

Backend Build Guide

From an empty folder to a production-grade authenticated API

Scope	Setting up a Node.js backend and implementing authentication end-to-end
Stack	Node 20 · TypeScript strict · Express · Prisma · PostgreSQL 16 · Redis · argon2 · JWT
Audience	Backend engineers picking up the Quantum Kaizen project
Outcome	An API the frontend can sign up, log in, refresh, MFA-enroll, and log out against
Format	

```
app.post("/auth/login", async (req, res) => {
  const { email, password, tenantCode } = req.body;
  const user = await svc.login(email, password, tenantCode);
  const { access, refresh } = await tokens.issue(user);
  res.cookie("qk_refresh", refresh, { httpOnly: true, secure: true });
  return res.json({ data: { user, accessToken: access } });
});
```

```
app.post("/auth/login", async (req, res) => {
  const { email, password, tenantCode } = req.body;
  const user = await svc.login(email, password, tenantCode);
  const { access, refresh } = await tokens.issue(user);
  res.cookie("qk_refresh", refresh, { httpOnly: true, secure: true });
  return res.json({ data: { user, accessToken: access } });
});
```

Contents

How to use this guide	4
Part 1 — Backend setup	4
Part 2 — Authentication: data model & primitives	11
Part 3 — Authentication: endpoints end-to-end	13
Part 4 — Hardening, tenancy, ops	21
Wrap-up: what you have now	24
Recommended reading order	24

How to use this guide

This guide is a linear, paste-as-you-go recipe. Each step has **why it exists** in prose and **what you type** as a code block. The reference implementation is the existing *server/* workspace in the quantumkaizen repo; every snippet is consistent with that codebase so you can cross-reference as you build.

Run each step against a fresh branch. Commit between steps so rollback is one command. The happy path assumes macOS or Linux; Windows users should run under WSL2 or adjust shell commands.

Prereqs Node 20 LTS, Git, Docker Desktop, a POSIX shell, and a text editor with TypeScript support.

Part 1 — Backend setup

Goal: by the end of Part 1, you have a booting Express server with Prisma wired to PostgreSQL, strict TypeScript, structured logs, environment-variable validation, and a green CI.

STEP 01 · Create the repository skeleton

Start with an npm workspaces monorepo so the backend can later share packages with the frontend.

BASH

```
mkdir qms && cd qms
git init
npm init -y
# Declare workspaces in the root package.json
npm pkg set workspaces='["server","client"]' --json
mkdir -p server/src client
```

STEP 02 · Bootstrap the server workspace

BASH

```
cd server
npm init -y
npm pkg set name='@qk/server' private=true
npm pkg set main='dist/index.js'
npm pkg set scripts.dev='tsx watch src/index.ts'
npm pkg set scripts.build='tsc'
npm pkg set scripts.start='node dist/index.js'
npm pkg set scripts.typecheck='tsc --noEmit'
npm pkg set scripts.test='vitest run'
```

STEP 03 · Install runtime + dev dependencies

BASH

```
# Runtime — web, auth, validation, db, logging
npm i express cors helmet cookie-parser compression hppmorgan \
  zod dotenv pino pino-pretty \
  bcryptjs jsonwebtoken \
  @prisma/client ioredis \
  express-rate-limit rate-limit-redis \
  argon2 otplib qrcode
```

```
# Dev – TypeScript, test, prisma CLI
npm i -D typescript @types/node @types/express @types/cors @types/cookie-parser \
  @types/bcryptjs @types/jsonwebtoken @types/morgan @types/qrcode \
  tsx vitest supertest @types/supertest \
  prisma eslint @typescript-eslint/eslint-plugin @typescript-eslint/parser
```

STEP 04 · Configure TypeScript (strict mode)

JSON

```
// server/tsconfig.json
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "lib": ["ES2022"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "forceConsistentCasingInFileNames": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "resolveJsonModule": true,
    "sourceMap": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

STEP 05 · Set up PostgreSQL + Redis via Docker

Keep the dev data in containers so every engineer has an identical environment.

YAML

```
# docker-compose.yml (repo root)
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: qms
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports: ['5432:5432']
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U postgres']
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine
```

```
ports: ['6379:6379']
volumes: [redisdata:/data]

volumes: { pgdata: {}, redisdata: {} }
```

BASH

```
docker compose up -d
```

STEP 06 · Initialise Prisma and the first model**BASH**

```
cd server
npx prisma init --datasource-provider postgresql
# Edit server/prisma/schema.prisma – add binary targets for Linux hosts.
```

PRISMA

```
// server/prisma/schema.prisma
generator client {
  provider      = "prisma-client-js"
  binaryTargets = ["native", "debian-openssl-3.0.x", "linux-musl-openssl-3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Tenant {
  id          String   @id @default(uuid())
  code        String   @unique
  name        String
  createdAt   DateTime @default(now())
  users       User[]
}

model User {
  id          String   @id @default(uuid())
  tenantId    String
  email       String
  name        String
  passwordHash String
  role        String   @default("TRAINEE")
  isActive    Boolean  @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  tenant Tenant @relation(fields: [tenantId], references: [id])

  @@unique([tenantId, email])
  @@index([tenantId])
}
```

STEP 07 · Create .env and run the first migration

BASH

```
# server/.env (never commit this)
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/qms?schema=public"
PORT=5000
NODE_ENV=development
JWT_SECRET="change-me-to-a-48-char-random-base64-value"
JWT_REFRESH_SECRET="different-48-char-random-base64-value"
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d
CORS_ORIGIN=http://localhost:3000
REDIS_URL=redis://localhost:6379
```

BASH

```
npx prisma migrate dev --name init
npx prisma generate
```

STEP 08 · Config loader with runtime validation

A typed *config* module is the single point that reads **process.env**. Every other file imports from here.

TS

```
// server/src/config/index.ts
import 'dotenv/config';
import { z } from 'zod';

const schema = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
  PORT: z.coerce.number().int().positive().default(5000),
  DATABASE_URL: z.string().url(),
  REDIS_URL: z.string().url().default('redis://localhost:6379'),
  JWT_SECRET: z.string().min(32),
  JWT_REFRESH_SECRET: z.string().min(32),
  JWT_ACCESS_EXPIRY: z.string().default('15m'),
  JWT_REFRESH_EXPIRY: z.string().default('7d'),
  CORS_ORIGIN: z.string().default('http://localhost:3000'),
});

const parsed = schema.safeParse(process.env);
if (!parsed.success) {
  console.error('[config] invalid env:', parsed.error.flatten().fieldErrors);
  process.exit(1);
}

export const config = parsed.data;
export default config;
```

STEP 09 · Structured logger (Pino)

TS

```
// server/src/lib/logger.ts
import pino from 'pino';
import config from '../../config/index.js';

export const logger = pino({
```

```
level: config.NODE_ENV === 'production' ? 'info' : 'debug',
// Pretty-printed in dev, JSON in prod so log shippers can parse it.
transport: config.NODE_ENV === 'development'
  ? { target: 'pino-pretty', options: { colorize: true, translateTime: 'HH:MM:ss' } }
  : undefined,
});
```

STEP 10 · Prisma client singleton + graceful shutdown

TS

```
// server/src/lib/prisma.ts
import { PrismaClient } from '@prisma/client';

const g = globalThis as unknown as { prisma?: PrismaClient };

export const prisma = g.prisma ?? new PrismaClient({
  log: ['warn', 'error'],
});

if (process.env.NODE_ENV !== 'production') g.prisma = prisma;

export async function disconnect() { await prisma.$disconnect(); }
```

STEP 11 · Express bootstrap with the essential middleware chain

TS

[illegible]


```
app.use(cors({
  origin: config.CORS_ORIGIN.split(',').map(s => s.trim()),
  credentials: true,
}));

// ■■ Parsing ████████████████████████████████████████████████████████████
app.use(express.json({ limit: '1mb' }));
app.use(express.urlencoded({ extended: true, limit: '1mb' }));
app.use(cookieParser());
app.use(compression());

// ■■ Health + API ████████████████████████████████████████████████████████████
app.get('/health', (_req, res) => {
  res.json({ status: 'ok', ts: new Date().toISOString() });
});
app.get('/health/ready', async (_req, res) => {
  try { await prisma.$queryRaw`SELECT 1`; res.json({ status: 'ok' }); }
  catch { res.status(503).json({ status: 'down' }); }
});

app.use('/api/v1', router);

// ■■ Error handler must be last ████████████████████████████████████████████████████████████
app.use(errorHandler);

http.listen(config.PORT, () => logger.info(`API on :${config.PORT} [${config.NODE_ENV}]`));

// Graceful shutdown
for (const sig of ['SIGTERM', 'SIGINT'] as const) {
  process.on(sig, async () => {
    logger.info(`${sig} received, shutting down`);
    http.close();
    await disconnect();
    process.exit(0);
  });
}
```

STEP 12 · Shared AppError + error handler

TS

```
// server/src/middleware/errorHandler.ts
import type { Request, Response, NextFunction } from 'express';
import { logger } from '../lib/logger.js';

export class AppError extends Error {
  constructor(
    public message: string,
    public statusCode = 500,
    public code = 'INTERNAL_ERROR',
    public details?: unknown,
  ) { super(message); }
}

export function errorHandler(err: unknown, req: Request, res: Response, _next: NextFunction) {
  if (err instanceof AppError) {
    if (err.statusCode >= 500) logger.error({ err, reqId: (req as any).id }, err.message);
    else logger.warn({ err, reqId: (req as any).id }, err.message);
  }
}
```

```

    return res.status(err.statusCode).json({
      error: { code: err.code, message: err.message, details: err.details },
    });
  }
  logger.error({ err, reqId: (req as any).id }, 'unhandled');
  return res.status(500).json({ error: { code: 'INTERNAL_ERROR', message: 'Unexpected error' } });
}

```

STEP 13 · Router skeleton

TS

```

// server/src/routes/index.ts
import { Router } from 'express';
import authRoutes from './auth.routes.js';

export const router = Router();

router.get('/', (_req, res) => res.json({ data: { name: 'QMS API', version: 'v1' } }));
router.use('/auth', authRoutes);

export default router;

```

STEP 14 · Smoke test

BASH

```

# From repo root
npm install          # installs workspace deps
cd server
npm run dev
# In another shell:
curl -s http://localhost:5000/health | jq

```

You should see { "status": "ok", "ts": "..." } — and the log line saying *"API on :5000 [development]"*. If not, check DATABASE_URL and the docker ps output.

STEP 15 · CI hook — keep it fast and boring

YAML

```

# .github/workflows/ci.yml
name: ci
on:
  push: { branches: [main] }
  pull_request: {}

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_PASSWORD: postgres

```

```

    POSTGRES_DB: qms
    options: >-
      --health-cmd="pg_isready -U postgres" --health-interval=5s
      --health-timeout=3s --health-retries=10
    ports: ['5432:5432']
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: '20', cache: 'npm' }
    - run: npm ci
    - run: cd server && npx prisma migrate deploy
      env: { DATABASE_URL: postgresql://postgres:postgres@localhost:5432/qms }
    - run: cd server && npm run typecheck
    - run: cd server && npm test

```

Part 2 — Authentication: data model & primitives

Goal: a correct set of tables and helpers before wiring up any endpoint. Auth bugs are mostly bad state machines; get the model right and the code falls out.

STEP 16 · Extend the Prisma schema for real auth

PRISMA

```

// server/prisma/schema.prisma (add to the existing file)

model RefreshToken {
  id          String    @id @default(uuid())
  userId      String
  familyId    String    // same for every token rotated in one login session
  tokenHash   String    @unique
  userAgent   String?
  ipAddress   String?
  expiresAt   DateTime
  revokedAt   DateTime?
  replacedById String?
  createdAt   DateTime @default(now())

  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([familyId])
}

model PasswordReset {
  id          String    @id @default(uuid())
  userId      String
  tokenHash   String    @unique
  expiresAt   DateTime
  usedAt      DateTime?
  createdAt   DateTime @default(now())
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
}

model MfaBackupCode {
  id          String    @id @default(uuid())

```

```

    userId    String
    codeHash   String
    usedAt     DateTime?
    createdAt  DateTime @default(now())
    user       User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  }

  // Add to User model:
  //   totpSecret      String?
  //   mfaEnrolledAt   DateTime?
  //   refreshTokens   RefreshToken[]
  //   passwordResets  PasswordReset[]
  //   backupCodes     MfaBackupCode[]

```

BASH

```
npx prisma migrate dev --name auth-primitives
```

STEP 17 · Password hashing (argon2id)

argon2id is the current best-practice password hash — memory-hard, tunable, recommended by OWASP.

TS

```

// server/src/auth/password.ts
import argon2 from 'argon2';

const options = {
  type: argon2.argon2id,
  memoryCost: 2 ** 16, // 64 MiB
  timeCost: 3,
  parallelism: 1,
};

export async function hashPassword(plain: string) {
  return argon2.hash(plain, options);
}

export async function verifyPassword(hash: string, plain: string) {
  try { return await argon2.verify(hash, plain); }
  catch { return false; }
}

// Re-hash on login if the stored hash was made with weaker params.
export function needsRehash(hash: string) {
  return argon2.needsRehash(hash, options);
}

```

STEP 18 · JWT helpers (access + refresh)**TS**

```

// server/src/auth/tokens.ts
import jwt, { type SignOptions } from 'jsonwebtoken';
import { createHash, randomBytes } from 'crypto';
import config from '../config/index.js';

export interface AccessClaims {

```

```

    sub: string;          // user id
    tenantId: string;
    role: string;
    email: string;
    // iat/exp added by jsonwebtoken
  }

export function signAccess(claims: AccessClaims) {
  const opts: SignOptions = { expiresIn: config.JWT_ACCESS_EXPIRY as SignOptions['expiresIn'] }
  ;
  return jwt.sign(claims, config.JWT_SECRET, opts);
}

export function verifyAccess(token: string) {
  return jwt.verify(token, config.JWT_SECRET) as AccessClaims;
}

// Refresh token = 48 random bytes, stored hashed, returned plaintext once.
export function mintRefreshToken(): { plain: string; hash: string } {
  const plain = randomBytes(48).toString('base64url');
  const hash = sha256(plain);
  return { plain, hash };
}

export function sha256(s: string) {
  return createHash('sha256').update(s).digest('hex');
}

```

STEP 19 · Rate-limiter (Redis-backed) — cheap but effective

TS

```

// server/src/middleware/rateLimit.ts
import rateLimit from 'express-rate-limit';
import RedisStore from 'rate-limit-redis';
import Redis from 'ioredis';
import config from '../config/index.js';

const redis = new Redis(config.REDIS_URL);

export const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 20,           // 20 attempts per IP per 15 min
  standardHeaders: true,
  legacyHeaders: false,
  store: new RedisStore({ sendCommand: (...a) => redis.call(...a) as any }),
  message: { error: { code: 'RATE_LIMITED', message: 'Too many auth attempts' } },
});

```

Part 3 — Authentication: endpoints end-to-end

Goal: every auth flow the frontend needs — sign up (optional), log in, refresh, logout, MFA enrollment and challenge, password reset. Each endpoint is a small number of lines because we did the grunt work in Part 2.

STEP 20 · Validation schemas (Zod)

TS

```
// server/src/auth/schemas.ts
import { z } from 'zod';

export const loginSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8).max(200),
  tenantCode: z.string().min(1),
  totpCode: z.string().length(6).optional(), // required if MFA enrolled
});

export const registerSchema = z.object({
  tenantCode: z.string(),
  email: z.string().email(),
  name: z.string().min(2),
  password: z.string().min(8).max(200),
});

export const refreshSchema = z.object({
  // refresh token is read from HttpOnly cookie, no body
});

export const forgotSchema = z.object({ email: z.string().email(), tenantCode: z.string() });
export const resetSchema = z.object({ token: z.string().min(20), newPassword: z.string().min(8) });
```

STEP 21 · Auth service — the business logic

TS

```
// server/src/auth/service.ts
import { prisma } from '../../lib/prisma.js';
import { AppError } from '../../middleware/errorHandler.js';
import { hashPassword, verifyPassword, needsRehash } from './password.js';
import { signAccess, mintRefreshToken, sha256 } from './tokens.js';
import { randomUUID } from 'crypto';

const REFRESH_DAYS = 7;

export async function login(email: string, password: string, tenantCode: string, ctx: { ua?: string; ip?: string }) {
  // 1. Find tenant + user
  const tenant = await prisma.tenant.findUnique({ where: { code: tenantCode } });
  if (!tenant) throw new AppError('Invalid credentials', 401, 'INVALID_CREDENTIALS');

  const user = await prisma.user.findUnique({
    where: { tenantId_email: { tenantId: tenant.id, email } },
  });
  if (!user || !user.isActive) throw new AppError('Invalid credentials', 401, 'INVALID_CREDENTIALS');

  // 2. Verify password (constant-time via argon2)
  const ok = await verifyPassword(user.passwordHash, password);
  if (!ok) throw new AppError('Invalid credentials', 401, 'INVALID_CREDENTIALS');

  // 3. Optionally rehash with stronger params
```

```

    if (needsRehash(user.passwordHash)) {
      const newHash = await hashPassword(password);
      await prisma.user.update({ where: { id: user.id }, data: { passwordHash: newHash } });
    }

    // 4. If MFA is enrolled, require totpCode – handled by caller; here we just assume verified.

    // 5. Issue tokens. Refresh token is one of a *family* so we can detect reuse.
    const familyId = randomUUID();
    const tokens = await issueTokens(user, familyId, ctx);

    return {
      user: publicUser(user, tenant),
      accessToken: tokens.access,
      refreshToken: tokens.refresh, // plaintext – returned to caller once
    };
  }

  export async function issueTokens(
    user: { id: string; tenantId: string; email: string; role: string },
    familyId: string,
    ctx: { ua?: string; ip?: string },
  ) {
    const access = signAccess({ sub: user.id, tenantId: user.tenantId, role: user.role, email: user.email });
    const { plain, hash } = mintRefreshToken();
    const expiresAt = new Date(Date.now() + REFRESH_DAYS * 24 * 60 * 60 * 1000);
    await prisma.refreshToken.create({
      data: {
        userId: user.id, familyId, tokenHash: hash, expiresAt,
        userAgent: ctx.ua?.slice(0, 255), ipAddress: ctx.ip?.slice(0, 45),
      },
    });
    return { access, refresh: plain };
  }

  export function publicUser(u: any, t: { id: string; code: string; name: string }) {
    return {
      id: u.id, tenantId: u.tenantId, email: u.email, name: u.name, role: u.role,
      tenant: { id: t.id, code: t.code, name: t.name },
    };
  }
}

```

STEP 22 · Refresh-token rotation + reuse detection

TS

```

// server/src/auth/service.ts (continued)
export async function rotateRefreshToken(plainFromCookie: string, ctx: { ua?: string; ip?: string }) {
  const hash = sha256(plainFromCookie);
  const existing = await prisma.refreshToken.findUnique({ where: { tokenHash: hash } });

  // Case 1: unknown token ⇒ probable theft attempt.
  if (!existing) throw new AppError('Invalid refresh token', 401, 'REFRESH_INVALID');

  // Case 2: token already replaced ⇒ reuse attack. Revoke the whole family.
}

```

```

    if (existing.replacedById || existing.revokedAt) {
      await prisma.refreshToken.updateMany({
        where: { familyId: existing.familyId, revokedAt: null },
        data: { revokedAt: new Date() },
      });
      throw new AppError('Refresh token reuse detected', 401, 'REFRESH_REUSED');
    }

    // Case 3: expired
    if (existing.expiresAt < new Date()) throw new AppError('Refresh token expired', 401, 'REFRESH_EXPIRED');

    // Case 4: all good – rotate.
    const user = await prisma.user.findUniqueOrThrow({ where: { id: existing.userId } });
    const { access, refresh } = await issueTokens(user, existing.familyId, ctx);
    const newRow = await prisma.refreshToken.findFirstOrThrow({
      where: { familyId: existing.familyId }, orderBy: { createdAt: 'desc' },
    });
    await prisma.refreshToken.update({
      where: { id: existing.id },
      data: { replacedById: newRow.id, revokedAt: new Date() },
    });
    return { access, refresh, user };
  }

  export async function logout(plainFromCookie: string) {
    const hash = sha256(plainFromCookie);
    const row = await prisma.refreshToken.findUnique({ where: { tokenHash: hash } });
    if (!row || row.revokedAt) return;
    await prisma.refreshToken.updateMany({
      where: { familyId: row.familyId, revokedAt: null },
      data: { revokedAt: new Date() },
    });
  }
}

```

STEP 23 · Auth controller — glue between HTTP and service

TS

```

// server/src/auth/controller.ts
import type { Request, Response, NextFunction } from 'express';
import { AppError } from '../middleware/errorHandler.js';
import { loginSchema, refreshSchema } from './schemas.js';
import * as svc from './service.js';
import config from '../config/index.js';

const REFRESH_COOKIE = 'qk_refresh';
const cookieOpts = {
  httpOnly: true,
  secure: config.NODE_ENV === 'production',
  sameSite: 'strict' as const,
  path: '/api/v1/auth',
  maxAge: 7 * 24 * 60 * 60 * 1000,
};

export async function login(req: Request, res: Response, next: NextFunction) {
  try {

```



```

    const body = loginSchema.parse(req.body);
    const result = await svc.login(body.email, body.password, body.tenantCode,
      { ua: req.get('user-agent') ?? undefined, ip: req.ip });
    res.cookie(REFRESH_COOKIE, result.refreshToken, cookieOpts);
    return res.json({ data: { user: result.user, accessToken: result.accessToken } });
  } catch (e) { next(e); }
}

export async function refresh(req: Request, res: Response, next: NextFunction) {
  try {
    refreshSchema.parse(req.body);
    const plain = req.cookies?.[REFRESH_COOKIE];
    if (!plain) throw new AppError('Missing refresh token', 401, 'REFRESH_MISSING');
    const { access, refresh, user } = await svc.rotateRefreshToken(plain,
      { ua: req.get('user-agent') ?? undefined, ip: req.ip });
    res.cookie(REFRESH_COOKIE, refresh, cookieOpts);
    return res.json({ data: { user: svc.publicUser(user, (user as any).tenant ?? {}), accessToken: access } });
  } catch (e) { next(e); }
}

export async function logout(req: Request, res: Response, next: NextFunction) {
  try {
    const plain = req.cookies?.[REFRESH_COOKIE];
    if (plain) await svc.logout(plain);
    res.clearCookie(REFRESH_COOKIE, { ...cookieOpts, maxAge: 0 });
    return res.json({ data: { ok: true } });
  } catch (e) { next(e); }
}

```

STEP 24 · Wire the routes + apply the rate-limiter

TS

```

// server/src/routes/auth.routes.ts
import { Router } from 'express';
import * as ctrl from '../auth/controller.js';
import { authLimiter } from '../middleware/rateLimit.js';

const r = Router();

r.post('/login', authLimiter, ctrl.login);
r.post('/refresh', authLimiter, ctrl.refresh);
r.post('/logout', ctrl.logout);

export default r;

```

STEP 25 · Auth middleware for protected routes

TS

```

// server/src/middleware/auth.ts
import type { Request, Response, NextFunction } from 'express';
import { AppError } from '../errorHandler.js';
import { verifyAccess } from '../auth/tokens.js';

declare module 'express-serve-static-core' {

```

```

interface Request {
  user?: { id: string; tenantId: string; role: string; email: string };
}

export function requireAuth(req: Request, _res: Response, next: NextFunction) {
  const raw = req.get('authorization');
  if (!raw?.startsWith('Bearer ')) throw new AppError('Missing token', 401, 'NO_AUTH');
  try {
    const claims = verifyAccess(raw.slice(7));
    req.user = { id: claims.sub, tenantId: claims.tenantId, role: claims.role, email: claims.email };
    next();
  } catch { throw new AppError('Invalid token', 401, 'BAD_TOKEN'); }
}

export function requireRole(...roles: string[]) {
  return (req: Request, _res: Response, next: NextFunction) => {
    if (!req.user || !roles.includes(req.user.role)) throw new AppError('Forbidden', 403, 'FORBIDDEN');
    next();
  };
}

```

STEP 26 · /auth/me — confirm the auth chain works

TS

```

// server/src/auth/controller.ts (append)
import { requireAuth } from '../middleware/auth.js';

export async function me(req: Request, res: Response, next: NextFunction) {
  try {
    const u = await prisma.user.findUniqueOrThrow({
      where: { id: req.user!.id },
      include: { tenant: { select: { id: true, code: true, name: true } } },
    });
    return res.json({ data: svc.publicUser(u, u.tenant) });
  } catch (e) { next(e); }
}

// server/src/routes/auth.routes.ts (append)
r.get('/me', requireAuth, ctrl.me);

```

BASH

```

# Happy-path test
TOKEN=$(curl -s -c /tmp/c.jar -XPOST localhost:5000/api/v1/auth/login \
  -H 'content-type: application/json' \
  -d '{"email":"admin@example.com","password":"Pass1234!","tenantCode":"DEMO"}' \
  | jq -r '.data.accessToken')
curl -s -H "Authorization: Bearer $TOKEN" localhost:5000/api/v1/auth/me | jq

```

STEP 27 · MFA — enroll (TOTP)

TS

```

// server/src/auth/mfa.ts

```

```

import { authenticator } from 'otplib';
import qrcode from 'qrcode';
import { prisma } from '../lib/prisma.js';
import argon2 from 'argon2';
import { randomBytes } from 'crypto';

export async function startEnrollment(userId: string, email: string) {
  const secret = authenticator.generateSecret();
  const otpauth = authenticator.keyuri(email, 'Quantum Kaizen', secret);
  const qrDataUri = await qrcode.toDataURL(otpauth);
  // Store provisional secret but not enrolledAt until the user verifies a code.
  await prisma.user.update({ where: { id: userId }, data: { totpSecret: secret } });
  return { otpauth, qrDataUri };
}

export async function confirmEnrollment(userId: string, code: string) {
  const user = await prisma.user.findUniqueOrThrow({ where: { id: userId } });
  if (!user.totpSecret) throw new Error('No enrollment in progress');
  const ok = authenticator.check(code, user.totpSecret);
  if (!ok) return { ok: false };

  // Generate 10 single-use backup codes, hashed in DB.
  const codes = Array.from({ length: 10 }, () => randomBytes(5).toString('hex'));
  await prisma.$transaction([
    prisma.user.update({ where: { id: userId }, data: { mfaEnrolledAt: new Date() } }),
    prisma.mfaBackupCode.deleteMany({ where: { userId } }),
    ...codes.map(c => prisma.mfaBackupCode.create({
      data: { userId, codeHash: hashSync(c) },
    })),
  ]);
  return { ok: true, backupCodes: codes };
}

function hashSync(plain: string): string {
  // Offload synchronous hashing to argon2 via top-level await in real code;
  // inline sync used here to keep the snippet short.
  return require('crypto').createHash('sha256').update(plain).digest('hex');
}

```

STEP 28 · MFA — challenge on login

TS

```

// server/src/auth/service.ts (augment login)
import { authenticator } from 'otplib';

// ... at the top of login(), after password verify but before issueTokens():
if (user.mfaEnrolledAt) {
  const code = (arguments as any)[0]?.totpCode; // pass through from controller
  if (!code) throw new AppError('MFA code required', 401, 'MFA_REQUIRED');
  if (!authenticator.check(code, user.totpSecret ?? '')) {
    // Optional: allow single-use backup codes here
    throw new AppError('Invalid MFA code', 401, 'MFA_BAD_CODE');
  }
}

```

Production note: emit MFA_REQUIRED without "invalid password" or "invalid MFA" distinction until both are verified, to avoid distinguishing failure reasons to an attacker.

STEP 29 · Password reset — request + confirm

TS

```
// server/src/auth/service.ts (append)
import { randomBytes } from 'crypto';
import { sendMail } from '../lib/mailer.js'; // you write this – thin wrapper over nodemailer

const RESET_TTL_MIN = 60;

export async function requestPasswordReset(email: string, tenantCode: string) {
  const tenant = await prisma.tenant.findUnique({ where: { code: tenantCode } });
  if (!tenant) return; // silent – don't leak tenant existence

  const user = await prisma.user.findUnique({ where: { tenantId_email: { tenantId: tenant.id, email } } });
  if (!user) return; // silent – don't leak user existence

  const plain = randomBytes(32).toString('base64url');
  const tokenHash = sha256(plain);
  const expiresAt = new Date(Date.now() + RESET_TTL_MIN * 60 * 1000);
  await prisma.passwordReset.create({ data: { userId: user.id, tokenHash, expiresAt } });

  const url = `${process.env.APP_URL}/reset-password?token=${plain}`;
  await sendMail(user.email, 'Reset your password',
    `Click to reset (valid ${RESET_TTL_MIN} minutes): ${url}`);
}

export async function confirmPasswordReset(token: string, newPassword: string) {
  const tokenHash = sha256(token);
  const row = await prisma.passwordReset.findUnique({ where: { tokenHash } });
  if (!row || row.usedAt || row.expiresAt < new Date())
    throw new AppError('Reset token invalid', 400, 'RESET_INVALID');

  const passwordHash = await hashPassword(newPassword);
  await prisma.$transaction([
    prisma.user.update({ where: { id: row.userId }, data: { passwordHash } }),
    prisma.passwordReset.update({ where: { id: row.id }, data: { usedAt: new Date() } }),
    // Invalidate existing sessions on password change – classic security hygiene.
    prisma.refreshToken.updateMany({
      where: { userId: row.userId, revokedAt: null },
      data: { revokedAt: new Date() },
    }),
  ]);
}
```

STEP 30 · Integration test (Supertest + Vitest)

TS

```
// server/src/auth/__tests__/auth.e2e.test.ts
import { describe, it, expect, beforeAll } from 'vitest';
import request from 'supertest';
import { app } from '../../index.js'; // export `app` from index.ts so it's importable
import { prisma } from '../../lib/prisma.js';
```

```
import { hashPassword } from '../password.js';

beforeAll(async () => {
  await prisma.user.deleteMany({});
  await prisma.tenant.deleteMany({});
  const tenant = await prisma.tenant.create({ data: { code: 'DEMO', name: 'Demo Co' } });
  await prisma.user.create({
    data: {
      tenantId: tenant.id, email: 'admin@example.com', name: 'Admin',
      role: 'TENANT_ADMIN', passwordHash: await hashPassword('Pass1234!'),
    },
  });
});

describe('auth', () => {
  it('logs in with correct creds and returns JWT + cookie', async () => {
    const r = await request(app).post('/api/v1/auth/login').send({
      email: 'admin@example.com', password: 'Pass1234!', tenantCode: 'DEMO',
    });
    expect(r.status).toBe(200);
    expect(r.body.data.accessToken).toMatch(/^eyJ/); // JWT header
    expect(r.headers['set-cookie'][0]).toMatch(/qk_refresh=/);
  });

  it('rejects wrong password with 401', async () => {
    const r = await request(app).post('/api/v1/auth/login').send({
      email: 'admin@example.com', password: 'nope', tenantCode: 'DEMO',
    });
    expect(r.status).toBe(401);
    expect(r.body.error.code).toBe('INVALID_CREDENTIALS');
  });
});
```

Part 4 — Hardening, tenancy, ops

By now auth works. Time to make it production-worthy: tenant isolation, audit trail, logging hygiene, shipping it.

STEP 31 · Tenant-scoped Prisma extension (defence layer 1)

Every query on a tenantId-owned table must include the current tenantId. Wrapping Prisma ensures you *cannot* forget.

TS

```
// server/src/lib/tenantPrisma.ts
import { prisma } from './prisma.js';

export function forTenant(tenantId: string) {
  return prisma.$extends({
    query: {
      $allModels: {
        async findMany({ args, query }) {
          args.where = { ...(args.where ?? {}), tenantId };
          return query(args);
        },
        async findFirst({ args, query }) {
```

```

    args.where = { ...(args.where ?? {}), tenantId };
    return query(args);
  },
  async updateMany({ args, query }) {
    args.where = { ...(args.where ?? {}), tenantId };
    return query(args);
  },
  async deleteMany({ args, query }) {
    args.where = { ...(args.where ?? {}), tenantId };
    return query(args);
  },
},
},
});
}

```

Use it as `req.db = forTenant(req.user.tenantId)` in a middleware, then call `req.db.document.findMany()` anywhere in that request.

STEP 32 · PostgreSQL Row-Level Security (defence layer 2)

SQL

```

-- server/prisma/migrations/20260101_rls/migration.sql
ALTER TABLE "User"    ENABLE ROW LEVEL SECURITY;
ALTER TABLE "Document" ENABLE ROW LEVEL SECURITY;
-- ...repeat for every tenant-scoped table.

CREATE POLICY tenant_isolation_user    ON "User"
  USING ("tenantId"::text = current_setting('app.tenant_id', true));

CREATE POLICY tenant_isolation_document ON "Document"
  USING ("tenantId"::text = current_setting('app.tenant_id', true));

```

Set the session variable from the request middleware:

TS

```

// server/src/middleware/tenantContext.ts
import type { Request, Response, NextFunction } from 'express';
import { prisma } from '../lib/prisma.js';

export async function applyTenantContext(req: Request, _res: Response, next: NextFunction) {
  if (!req.user) return next();
  await prisma.$executeRawUnsafe(`SELECT set_config('app.tenant_id', $1, true)`, req.user.tenantId);
  next();
}

```

STEP 33 · Audit log for every state change

PRISMA

```

model AuditLog {
  id          String    @id @default(uuid())
  tenantId    String
  userId      String?
  action      String     // "document.create", "capa.approve", "auth.login"
  entityType  String?
}

```

```

    entityId    String?
    beforeState Json?
    afterState  Json?
    ipAddress   String?
    userAgent   String?
    timestamp   DateTime @default(now())
    hash        String    // sha256(prev.hash + this record) – chain for tamper-evidence

    @@index([tenantId, timestamp])
    @@index([tenantId, entityType, entityId])
  }

```

Write one row per mutating action, chained by **hash** so any later tamper is detectable during validation audits.

STEP 34 · Security headers audit + secret rotation SOP

- Run **npm audit --production** weekly in CI; block merge on criticals.
- Snyk or Dependabot on the GitHub repo; auto-PR minor bumps.
- JWT secrets rotated quarterly; old secrets kept on the verification key list for 24 h during rotation.
- HTTPS-only cookies; SameSite=strict on the refresh cookie; Secure flag in production.
- CSP with default-src 'self'; explicit allowlist for API, fonts, analytics.
- Principle of least privilege in the DB: app user has no DROP / CREATE; migration runner has a separate role.

STEP 35 · Production deploy — entrypoint + Render

BASH

```

#!/usr/bin/env bash
# server/start.sh
set -e
npx prisma migrate deploy --schema prisma/schema.prisma
if [ "${SEED_ON_START}" = "1" ]; then
  npx tsx prisma/seed.ts || echo 'seed failed, continuing'
fi
exec node dist/index.js

```

YAML

```

# render.yaml (excerpt)
services:
  - type: web
    name: qk-api
    runtime: node
    buildCommand: npm ci --no-audit && npm run build --workspace=server
    startCommand: cd server && ./start.sh
    healthCheckPath: /health
    envVars:
      - key: DATABASE_URL
        fromDatabase: { name: qk-db, property: connectionString }
      - key: JWT_SECRET
        generateValue: true
      - key: JWT_REFRESH_SECRET
        generateValue: true
      - key: CORS_ORIGIN
        value: https://app.your-domain.example
databases:

```

```
- name: qk-db
  postgresMajorVersion: '16'
```

Wrap-up: what you have now

- A booting Node + Express + TypeScript API wired to PostgreSQL and Redis.
- Strict TS with per-env config, Pino logs, a shared AppError / error handler, and a health endpoint.
- argon2id password hashing + rehash-on-login, JWT access tokens (15m), rotating refresh tokens with reuse detection.
- Login, refresh, logout, /auth/me, MFA enrollment + challenge, password-reset flow.
- Rate limiting on auth endpoints, security headers via Helmet, HSTS, strict CORS.
- Tenant scoping via Prisma extension + Postgres Row-Level Security as defence in depth.
- Audit-log skeleton ready for every mutating endpoint.
- CI that migrates a fresh Postgres, type-checks, and runs integration tests.
- A clear deploy path to Render with a minimal entrypoint that applies migrations.

Next tracks (1) RBAC + permission catalogue on top of **role**. (2) Dynamic workflow engine driven by JSON definitions. (3) Dynamic form engine sharing Zod schemas with the UI. (4) SSO via SAML 2.0 + OIDC. Each is its own document — this guide intentionally stopped at the authenticated foundation so Part 1–4 stays short enough to read in one sitting.

Recommended reading order

- *Designing Data-Intensive Applications* — Chapter 5 (replication) & 9 (consistency).
- OWASP ASVS v4 — chapters 2 (authentication) & 3 (session management).
- RFC 6749 (OAuth 2.0) + RFC 6238 (TOTP) + RFC 7519 (JWT).
- Prisma docs — "Extending Prisma Client" and "Connection management".